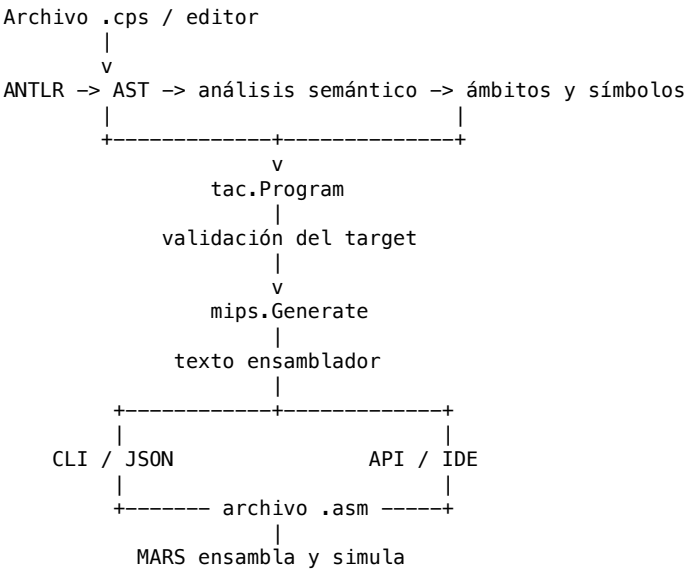
Arquitectura de Compiscript MIPS

Índice

1. Pipeline y fronteras 1
2. Módulos y responsabilidades 1
3. Representación de valores 2
4. Homes, registros y spilling 3
5. Activación y secuencia de llamada 3
6. Entornos, capturas y métodos 4
7. Objetos, listas y guardas 4
8. Excepciones y restauración 4
9. Frontera IDE y compatibilidad 4

El diseño conserva el frontend y el TAC de la fase anterior. La novedad es una traducción física separada a MIPS, acompañada por helpers de ejecución también escritos en ensamblador. MARS es un consumidor del resultado, no una fase que el servidor ejecute durante la compilación.

1. Pipeline y fronteras



`CompileMIPS` llama a `Compile` y solo agrega la salida de target. Si hay errores anteriores, no intenta generar MIPS. Si falla el backend, devuelve `MIPS_GENERATION` en fase `codegen`, sin `mipsText` parcial; los errores ubicados conservan el span de la instrucción TAC.

La generación consume estructuras: opcodes, operandos tipados, procedimientos y layouts. La impresión textual del TAC es una interfaz de lectura, no un formato que se vuelva a parsear.

2. Módulos y responsabilidades

Ruta	Responsabilidad
`internal/compiscript/frontend/`	ANTLR y construcción del AST
`internal/compiscript/semantic/`	Resolución, tipos y snapshots de ámbitos

Ruta	Responsabilidad
`internal/compiscript/tac/`	IR, control, funciones, entornos y layouts abstractos
`internal/compiscript/compile.go`	Fachada TAC previa, preservada
`internal/compiscript/mips_compile.go`	Fachada aditiva de MIPS y diagnósticos de target
`mips/generate.go`	Datos/literales, nombres físicos, homes, frames y despacho de instrucciones
`mips/scalars.go`, `mips/runtime.go`	Operaciones, guardas escalares y helpers MIPS comunes
`mips/calls.go`, `mips/closures.go`	Argumentos, llamadas, entornos retenidos y clausuras
`mips/references.go`	Objetos, herencia, métodos, listas, descriptores y render de referencias
`mips/exceptions.go`	Validación de flujo de handlers y desenrollado dinámico
`cmd/compiscript-mips/`, `cmd/ide/mips.go`	Interfaces de producto; no implementan otro compilador
`web/app.js`	Presentación como texto y control de vigencia/descarga

Las rutas `mips/...` de la tabla son relativas a `internal/compiscript/`. Las pruebas de ejecución usan MARS externo; los oráculos TAC existentes permanecen únicamente en pruebas.

3. Representación de valores

Las variables y temporales almacenan punteros de cuatro bytes. El valor apuntado empieza en una caja alineada a ocho bytes, normalmente de 16 bytes; objetos y listas pueden ocupar más.

Etiqueta	Valor	Payload principal
1	Entero	Double binary64 en offset 8
2	Float	Double binary64 en offset 8
3	Booleano	Cero/uno en offset 4
4	String	Puntero a bytes en 4; longitud UTF-8 en 8
5	Objeto	Descriptor de clase en 4; vector de entornos en 8; campos desde 16
6	Lista	Descriptor de elemento en 4; longitud en 8; elementos desde 16
7	Función/método	Código en 4; entorno en 8; receptor en 12
8	Null	Caja singleton, sin instancia detrás
9	Excepción	Texto en 4; longitud en bytes en 8; identidad por caja

Los números enteros del lenguaje no se reducen a una palabra con signo de 32 bits: su payload sigue la política binary64 del oráculo. La etiqueta mantiene la distinción entero/float para guardas y promociones.

Una caja privada con etiqueta cero representa un binding declarado sin valor. No es un valor público ni `null`. Un puntero cero en una celda reservada indica que todavía no se activó esa declaración. Una dependencia de llamada exige declaración activada; una lectura exige además un valor definido. Los campos de objetos no inicializados usan celda cero y tienen su propia guarda.

Los escalares se tratan como valores de cajas; objetos y listas comparten una asignación mutable entre alias. La conversión entero a float crea otra caja: no cambia un literal ni retaggea un valor que otro alias conserva.

4. Homes, registros y spilling

Cada resultado TAC se escribe inmediatamente en su home canónico. En el programa principal puede haber celdas estáticas; las activaciones tienen ranuras para operandos propios. Cuando existen layouts de entornos, los símbolos se resuelven a sus celdas léxicas retenidas. Los temporales continúan siendo propios del frame y se pueden reutilizar con etiquetas distintas.

Registro	Papel en este backend
`$sp`	Pila de frames, vectores de argumentos y stacks auxiliares
`$fp`	Base estable del frame actual
`$ra`	Continuación de llamada MIPS
`$s0`	Entorno léxico capturado
`$s1`	Cabeza de entradas de entorno propias de la activación
`$s2`–`$s7`	Operand/result scratch preservado entre helpers y llamadas
`$t0`–`$t7`	Scratch por instrucción; `getReg` presta registros de este conjunto
`$t8`, `$t9`	Scratch reservado para tareas como alineación/continuación/dispatch
`$a0`–`$a2`	Argumentos del ABI privado y de helpers/syscalls
`$v0`	Resultado de función o helper, y selector/resultado de syscall
`$f...`	Cálculos y comparaciones de doubles

Los helpers conservan `$s0`–`$s7` y `$fp` salvo que el contrato de una operación indique el resultado explícito. Pueden modificar `$a`, `$t`, `$v` y registros FPU. Los valores que deben vivir más allá de una instrucción ya están en memoria.

No se afirma una ABI o32 interoperable con bibliotecas del sistema. Tampoco hay asignación global optimizada ni coloreo de grafos. La pila y el spill inmediato son la estrategia principal de corrección.

5. Activación y secuencia de llamada

El tamaño físico del frame se redondea a múltiplo de ocho. Esto no debe confundirse con afirmar que el `$sp` inicial de toda configuración MARS sea idéntico o que se implemente la ABI de un sistema operativo.

Offset respecto de `$fp`	Contenido
0	`$ra` del caller
4	`$fp` anterior
8 a 36	`$s0` a `$s7`, una palabra cada uno
40 a 47	Reserva del encabezado
Desde 48	Homes de operandos propios, cuatro bytes por puntero

Los slots abstractos de `tac.FrameLayout` no se convierten ciegamente en bytes multiplicando todo por cuatro. El backend asigna homes a los operandos y agrega su encabezado físico propio.

Caller

1. Verifica las dependencias de bindings y selecciona el entorno/callee correspondiente.
2. Reserva un vector de argumentos en la pila, redondeado a ocho bytes.
3. Lee los argumentos ya evaluados por el TAC y guarda sus punteros en ese vector.
4. Pasa dirección del vector en `$a0`, entorno en `$a1` y receptor en `$a2`; usa `jal` o `jalr`.
5. Recupera el espacio del vector y guarda `$v0` en el home del destino, si existe.

Callee y retorno

1. Reserva su frame y preserva continuación, frame anterior y registros guardados.
2. Copia todos los argumentos a homes propios antes de validar valores o entrar al entorno.
3. Verifica parámetros/dependencias y ejecuta el TAC traducido.
4. Deja el puntero del resultado en `$v0` —o null para un procedimiento sin resultado—.
5. Restaura registros, `$sp`, `$fp` y `$ra`, y retorna con `jr`.

No hay un límite de cuatro argumentos por usar `$a0`–`$a3`: el ABI pasa un vector por pila. Sí existe un límite explícito de desplazamientos del vector. Cada invocación tiene homes propios, por lo que la recursión no pisa los locales del caller.

6. Entornos, capturas y métodos

Un entorno retenido contiene: ID de ámbito en 0, padre léxico en 4, entrada propia anterior en 8 y celdas de bindings desde 12. `EnterEnvironment` asigna una instancia nueva cada vez que se ejecuta, incluso al reentrar a un bloque en un ciclo.

La búsqueda prioriza las instancias propias más recientes (`$s1`) y después la cadena capturada (`$s0`). Una clausura retiene la instancia concreta de su ámbito de definición. Dos clausuras hermanas comparten una celda; dos invocaciones distintas del mismo ámbito no se confunden por tener el mismo ID estático.

Los objetos conservan también los entornos vigentes para su clase y ancestros. Seleccionar un método más tarde no debe capturar por accidente otro entorno con el mismo layout. Un método ligado es la tupla código/entorno/receptor; esa tupla determina su igualdad.

7. Objetos, listas y guardas

Los campos heredados mantienen offsets estables. La selección virtual usa la clase efectiva del objeto; una selección explícita de inicializador no es un dispatch virtual. El TAC ya ordena inicializadores y constructor: `NewObject` no vuelve a ejecutarlos por su cuenta.

Las listas conservan descriptor del elemento y referencias a sus valores. La lectura/escritura valida receiver, null, tipo de índice, finitud, integridad y límites antes de convertir el índice a entero MIPS. Una comprobación de tipo conserva la identidad de la lista, incluida una lista genérica vacía; las vistas nominales y anidadas usan descriptores y guardas.

Los strings guardan bytes y longitud. La salida por syscall 15 preserva UTF-8 y NUL embebido; una syscall de string NUL-terminado no sería equivalente. La impresión de referencias es estable por ID; las listas limitan presupuesto/profundidad y detectan ancestros cíclicos.

8. Excepciones y restauración

Los handlers forman una cadena dinámica, independiente de dónde se definió una clausura. Cada registro de 56 bytes conserva handler anterior, destino catch, `$sp`, `$fp`, `$ra`, `$s0`–`$s7` y excepción pendiente previa.

Un fallo de lenguaje llega a `rt_fault` y `rt_raise`. Si existe handler, se crea una excepción etiquetada, se retira el handler **antes** del catch y se restauran los punteros/registros guardados. Así se abandonan frames y stacks auxiliares intermedios sin usar sus continuaciones.

Restaurar la cabeza de entornos vuelve a las instancias visibles al instalar el handler; no revierte mutaciones anteriores ni destruye capturas escapadas. `PopHandler` respeta las limpiezas explícitas del TAC en return/break/continue. Metadata o flujo de handlers inválidos se rechazan como infraestructura, no se convierten en una excepción capturable del fuente.

9. Frontera IDE y compatibilidad

`POST /api/compiscript/mips` acepta el mismo transporte estricto de fuente que Analyze/TAC y solo genera texto. La UI usa nodos de texto para el ensamblador; editar, recibir error o invalidar una respuesta tardía elimina la descarga anterior. La compilación TAC y el AST permanecen separados.

No se agrega ejecución arbitraria en el servidor. El usuario descarga `.asm` y lo ejecuta con MARS local. La guía, la referencia y la evidencia completan comandos, ejemplos, límites y resultados observados.